



Unconditional Branching

The second type of **Control Instructions** is **Branching Instructions**, which are general instructions that allow us to **jump** to any point in the program if a specific condition is met. Let's first discuss the most basic branching instruction: **jump**, which will always jump to a location unconditionally.

JMP loopFib

The **jump** instruction jumps the program to the label or specified location in its operand so that the program's execution is continued there. Once a program's execution is directed to another location, it will continue processing instructions from that point. If we wanted to temporarily jump to a point and then return to the original calling point, we would use functions, which we will discuss in the next section.

The basic **jump** instruction is unconditional, which means that it will always jump to the specified location, regardless of the conditions. This contrasts with **Conditional Branching** instructions that only jump if a specific condition is met, which we'll discuss next.

Instruction	Description	Example
<code>jmp</code>	Jumps to specified label, address, or location	<code>jmp loop</code>

Let's try using `jmp` in our `fib.s` program, and see how it would change the execution flow. Instead of looping back to `loopFib`, let's `jmp` there instead: So, our final code is:

```
nasm
global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
    mov rcx, 10
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    jmp loopFib
```

Now, let's assemble our code, and run it with `gdb`. We'll once again `b loopFib`, and see how it changes:

```
$ ./assembler.sh fib.s -g
gef> b loopFib
Breakpoint 1 at 0x40100e
gef> r
```

```
registers — $rbx : 0x1
```

```
$rcx : 0xa
```

```
$rcx : 0xa
```

```
registers — $rax : 0x1
```

```
$rbx : 0x1
```

```
$rcx : 0xa
```

```
registers — $rax : 0x1
```

```
$rbx : 0x2
```

```
$rcx : 0xa
```

```
registers — $rax : 0x2
```

```
$rbx : 0x3
```

```
$rcx : 0xa
```

```
registers — $rax : 0x3
```

```
$rbx : 0x5
```

```
$rcx : 0xa
```

```
registers — $rax : 0x5
```

```
$rbx : 0x8
```

```
$rcx : 0xa
```

We press `c` a few times to let the program jump multiple times back to `loopFib`. As we can see, the program is still performing the same function and still correctly calculating the Fibonacci Sequence. However, the main difference from the loop is that 'rcx' is not decrementing. This is because a `jmp` instruction does not consider `rcx` as a counter (like `loop` does), and so it will not automatically decrement it.

Let's delete our break point with `del 1`, and press `c` to see to what end the program will run:

```
gef> info break
```

Num	Type	Disp	Enb	Address	What
-----	------	------	-----	---------	------

```
1 breakpoint keep y 0x000000000040100e <loopFib>
breakpoint already hit 6 times
gef> del 1
gef> c
Continuing.
```

```
Program received signal SIGINT, Interrupt.
0x000000000040100e in loopFib ()
```

```
registers — $rax : 0x2e02a93188557fa9
$rbx : 0x903b4b15ce8cedf0
$rcx : 0xa
```

We noticed that the program kept running until we pressed `ctrl+c` after a few seconds to kill it, by which point the Fibonacci number has reached `0x903b4b15ce8cedf0` (which is a huge number). This is because of the unconditional `jmp` instruction, which keeps jumping back to `loopFib` forever since a specific condition does not restrict it. This is similar to a `(while true)` loop.

This is why unconditional Branching is usually used in cases where need always to jump, and it is not suitable for loops, as it will loop forever. To stop jumping when a specific condition is met, we will use `Conditional Branching` for our next steps.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

 +1

 +40



Try to jump to "func" before "loop loop". What is the hex value of "rbx" at the end?

Write your answer

 Submit

 Hint

 Download File

← Previous

Section 14 / 24

 +20

Next

Mark Complete & Next